Отчёт по практике 3

Перебор паролей по хэшу: свой инструмент и сравнение с John the Ripper

Выполнил: Колесников Андрей Юрьевич, 4 курс, группа БСБО-03-23, студенческий билет 23Б0099, номер по журналу 10

РТУ МИРЭА, Институт кибербезопасности и цифровых технологий

1. Цель работы

Цель работы — написать собственный переборщик паролей по хэшу, проверить его на учебных хэшах из методички, сравнить с профессиональной утилитой и оценить, как на время подбора влияют алгоритм хеширования и сложность пароля. Дополнительно нужно было ускорить программу за счёт многопоточности или многопроцессности.

2. Исходные данные и условия эксперимента

В работе использованы ровно те 16 хэшей, которые даны в методичке: по четыре для SHA-1, MD5, bcrypt (cost = 10) и Argon2id (m = 65536 КБ, t = 3, p = 2). Исходные пароли заранее не были известны, подбор шёл только по этим учебным хэшам. Никакие чужие хэши, учётные записи или реальные системы не затрагивались.

Замеры выполнены на ноутбуке: AMD Ryzen 7 5700U (8 физических ядер, 16 логических потоков), 16 ГБ оперативной памяти, Windows 11, Python 3.12.4. Основной прогон лабораторной запускался с восемью рабочими процессами — по числу физических ядер, чтобы не мешать измерениям гиперпоточностью. В сравнении с John the Ripper это надо учитывать: John в тех же условиях запускался на 16 потоках OpenMP, если формат это поддерживает.

3. Устройство собственного переборщика

Основная программа — bruteforce.py. Она принимает хэш, сама определяет алгоритм по его формату и перебирает кандидатов либо из словаря, либо по маске. Поддерживаются SHA-1, MD5, bcrypt и Argon2id — то есть все четыре алгоритма из задания. Для bcrypt стоимость берётся из самого хэша ($2a$10$...), для Argon2id параметры m, t и p тоже читаются из строки хэша, поэтому отдельно указывать их не нужно.

3.1. Определение алгоритма и проверка хэша

Алгоритм определяется в функции detect_algorithm: префикс $argon2 соответствует Argon2id, префиксы $2a$, $2b$ и $2y$ — bcrypt, строка из 40 шестнадцатеричных

символов — SHA-1, из 32 — MD5. Если формат не подошёл, программа просит указать тип вручную через --type. Перед запуском перебора validate_target ещё раз проверяет длину и корректность строки: иначе опечатка в хэше выглядела бы как «пароль не найден», хотя на самом деле неверны входные данные.

3.2. Проверка одного кандидата

Для SHA-1 и MD5 используется hashlib. Сравниваются сразу байты digest, а не шестнадцатеричные строки: так на каждый кандидат не тратится время на перевод хэша в hex. Для bcrypt вызывается bcrypt.checkpw, для Argon2id — argon2.low_level.verify_secret с типом Type.ID. Ошибка «пароль не подошёл» обрабатывается отдельно от ошибки формата: в первом случае кандидат просто пропускается, во втором программа сообщает о проблеме с хэшем.

3.3. Словарная атака и перебор по маске

Словарная атака читает wordlist.txt построчно и проверяет каждое слово. В словаре 241 распространённый пароль: цифровые последовательности, простые английские слова, несколько вариантов с заглавной буквы и типичные связки вроде qwerty. Это небольшой самодельный список, а не промышленный словарь на миллионы строк, но для учебных хэшей его хватило, чтобы увидеть разницу между алгоритмами.

Перебор по маске — это уже настоящий полный перебор, а не словарь. Маска записывается в привычном виде: ?l — строчные латинские буквы, ?u — заглавные, ?d — цифры, ?s — спецсимволы, ?a — все перечисленные вместе, ?? — сам знак вопроса. Например, ?d?d?d?d?d?d означает все числа от 000000 до 999999, то есть ровно миллион вариантов. Разбор маски находится в parse_mask, а генератор вариантов — в iter_mask.

3.4. Многопроцессность

Перебор распараллелен через ProcessPoolExecutor, а не через threading. Причина простая: hashlib в CPython не отпускает GIL, поэтому несколько потоков внутри одного процесса будут считаться по очереди и ускорения не дадут. Процессы же выполняются по-настоящему параллельно на разных ядрах. Главный процесс режет последовательность кандидатов на пачки и держит в работе примерно два пачки на каждый процесс, чтобы ядра не простаивали. Как только одна пачка вернула пароль, остальные задачи отменяются, и программа не досчитывает уже ненужные варианты.

Размер пачки зависит от стоимости одного хэша. Для SHA-1 и MD5 это 50 000 кандидатов: хэш считается очень быстро, и передавать задания мелкими порциями было бы накладно. Для bcrypt пачка равна 16, для Argon2id — 8: там один хэш занимает десятки миллисекунд, и большая пачка после находки ещё долго досчитывалась бы впустую. По той же причине число процессов для Argon2id по умолчанию ограничено четырьмя: каждый процесс держит в памяти 64 МБ, и шестнадцать таких процессов съели бы целый гигабайт.

3.5. Запуск

Проверить один хэш по словарю:

python bruteforce.py --hash 7c4a8d09ca3762af61e59520943dc26494f8941b --dict wordlist.txt

Перебрать шесть цифр по маске:

python bruteforce.py --hash e10adc3949ba59abbe56e057f20f883e --mask "?d?d?d?d?d?d" --workers 4

Полный лабораторный прогон по всем 16 хэшам:

python bruteforce.py --lab --workers 8

Есть также --selftest: шесть быстрых проверок на эталонных значениях SHA-1 и MD5 для строки abc, разбор маски, проверка одного кандидата, подбор двумя процессами, bcrypt и Argon2id. Он нужен, чтобы перед сдачей убедиться, что программа вообще правильно считает хэши, а не просто выдаёт «не найдено».

4. Методика замеров

Словарная атака прогонялась по всем 16 хэшам одним запуском bruteforce.py --lab. Для каждого хэша заново создавался пул процессов, поэтому в измеренное время входит и запуск восьми процессов. На коротком словаре из 241 слова это заметная доля, особенно для SHA-1 и MD5, где сам перебор занимает доли миллисекунды. Для bcrypt и Argon2id накладные расходы уже не важны: там время определяется вычислениями.

Отдельно по маске ?d?d?d?d?d?d перебирались лёгкие хэши SHA-1 и MD5. Ограничение --max-candidates 1000000 как раз соответствует полному множеству шестизначных чисел, так что это честный полный перебор, а не первые сто тысяч вариантов.

Для сравнения с John the Ripper собран свежий jumbo-образ в Docker: готовый пакет из Debian оказался обычной версией без нужных форматов, а старый образ с Docker Hub не понимал Argon2id. John запускался на тех же четырёх хэшах каждого алгоритма и с тем же wordlist.txt. В таблицу попадает внутреннее время John из строки DONE, а не полное время вместе с запуском контейнера: запуск Docker добавляет ещё около 2,5 секунды и к скорости перебора отношения не имеет.

Ускорение от числа процессов измерялось отдельным скриптом parallel_benchmark.py. Там специально взяты цели, которых нет в перебираемом множестве: иначе замер оборвался бы на первом совпадении и показывал не полную скорость, а время до случайной находки. Для MD5 перебирался миллион шестизначных чисел, для bcrypt и Argon2id — все 241 слово по сложному хэшу из методички. Каждый вариант запускался на 1, 2, 4 и 8 процессах.

5. Результаты словарной атаки по 16 хэшам

Из 16 хэшей словарём удалось восстановить 12. Во всех четырёх алгоритмах нашлись лёгкий, средний и «очень сложный» пароли, а «сложный» не нашёлся нигде. Это не ошибка классификации: лёгкий пароль оказался числом 123456, средний — словом apple, «очень сложный» — словом Create, которое просто входит в словарь. А «сложный» пароль в словаре отсутствует, поэтому его не взял ни один алгоритм. Отсюда сразу видно, что словарная атака зависит в первую очередь от содержимого словаря, а не от названия категории сложности в методичке.

Алгоритм	Сложность	Пароль	Позиция в словаре	Время, с
SHA-1	лёгкий	123456	1	0,36
SHA-1	средний	apple	103	0,32
SHA-1	сложный	не найден	—	0,32
SHA-1	очень сложный	Create	104	0,31
MD5	лёгкий	123456	1	0,30
MD5	средний	apple	103	0,32
MD5	сложный	не найден	—	0,33
MD5	очень сложный	Create	104	0,34
bcrypt	лёгкий	123456	1	0,57
bcrypt	средний	apple	103	1,30
bcrypt	сложный	не найден	—	4,29
bcrypt	очень сложный	Create	104	1,43
Argon2id	лёгкий	123456	1	0,89
Argon2id	средний	apple	103	8,40
Argon2id	сложный	не найден	—	14,71
Argon2id	очень сложный	Create	104	7,80

По времени уже видна разница между алгоритмами. Для SHA-1 и MD5 все четыре хэша вместе заняли около 1,3 секунды, причём почти всё это — запуск процессов и чтение словаря. Для bcrypt суммарное время составило 7,6 секунды, для Argon2id — 31,8 секунды. При этом bcrypt и Argon2id находят пароль в начале словаря за доли секунды, но каждый проверенный кандидат стоит десятки миллисекунд, поэтому даже маленький словарь ощутим.

6. Полный перебор по маске

Для лёгких хэшей SHA-1 и MD5 был выполнен полный перебор всех шестизначных чисел. Пароль 123456 находится в позиции 123457, а не 123456, потому что нумерация идёт с единицы: вариант 000000 — первый, 000001 — второй, и так далее.

Алгоритм	Маска	Пароль	Позиция	Время, с
SHA-1	?d?d?d?d?d?d	123456	123 457	0,84

MD5	?d?d?d?d?d?d	123456	123 457	0,77

Миллион вариантов SHA-1 перебрался за 0,84 секунды, MD5 — за 0,77 секунды. В эти числа входит запуск восьми процессов. В отдельном замере на миллион вариантов MD5 с восемью процессами получилось 1,35 секунды и 740 тысяч хэшей в секунду, то есть установившаяся скорость выше, чем в коротком лабораторном прогоне. Для шестизначного пароля это означает, что на быстром хэше он подбирается практически мгновенно, и длина сама по себе не спасает, если алфавит маленький.

7. Ускорение за счёт нескольких процессов

Результаты parallel_benchmark.py приведены в таблице. Для каждого алгоритма показано время полного прохода на 1, 2, 4 и 8 процессах и ускорение относительно одного процесса.

Алгоритм	1 процесс	2 процесса	4 процесса	8 процессов	Ускорение
MD5	3,49	1,94	1,62	1,35	2,59x
bcrypt	29,38	17,42	8,78	4,87	6,04x
Argon2id	51,47	28,30	20,02	17,14	3,00x

Лучше всего масштабируется bcrypt: 6,04 раза на восьми процессах. Один хэш bcrypt считается около 80 миллисекунд, поэтому накладные расходы на передачу заданий почти не заметны, и процессы загружены равномерно. MD5 ускорился только в 2,59 раза. Здесь ограничение не в процессоре, а в главном процессе: он один генерирует миллион строк и раскладывает их по пачкам, а потом сериализует и отправляет дочерним процессам. Когда сам хэш считается за доли микросекунды, эта работа становится узким местом.

Argon2id дал ускорение 3,00 раза. Причина в памяти: каждый процесс занимает 64 МБ и постоянно читает и пишет их, поэтому восемь процессов упираются в пропускную способность памяти, а не в ядра. Это как раз тот эффект, ради которого Argon2 делали memory-hard: увеличивать число параллельных попыток становится дорого не только по процессорному времени, но и по памяти.

8. Сравнение с John the Ripper

John запускался на тех же четырёх хэшах каждого алгоритма и с тем же словарём. В таблице для моего инструмента указана сумма времени по четырём отдельным запускам, для John — его внутреннее время на все четыре хэша сразу.

Алгоритм	Мой инструмент, 4 хэша	John, 4 хэша	Потоков у John	Найдено
SHA-1	1,31	< 1	1	3 из 4
MD5	1,29	< 1	1	3 из 4

bcrypt	7,60	4	16	3 из 4
Argon2id	31,79	13	16	3 из 4

На быстрых хэшах сравнивать время бессмысленно: словарь слишком короткий, и оба инструмента заканчивают раньше, чем успевают показать устойчивую скорость. У John время в строке DONE округлено до секунды, поэтому там и стоит «< 1».

На медленных алгоритмах John оказался быстрее: 4 секунды против 7,6 на bcrypt и 13 секунд против 31,8 на Argon2id, то есть примерно в 1,9 и 2,4 раза. Разница объясняется условиями, а не ошибкой в моей программе. John считает все четыре хэша одновременно и использует 16 потоков OpenMP, а мой переборщик запускает по одному хэшу за раз и на восьми физических ядрах. Плюс John написан на C и использует оптимизированные AVX2-реализации. Если запустить мой код на 16 процессах, разрыв должен сократиться, но полностью не исчезнет. Главное, что оба инструмента нашли одни и те же три пароля из четырёх и не взяли «сложный» — значит, реализация считает правильно.

9. Влияние алгоритма и сложности пароля

Для сравнения самих алгоритмов отдельно замерялась скорость одного ядра на одном хэше (hash_benchmark.py). Эти числа не зависят от запуска процессов и лучше показывают стоимость алгоритма.

Алгоритм	Средняя скорость, хэш/с	Время одного хэша
SHA-1	2 233 799	0,000448 мс
MD5	2 111 364	0,000474 мс
bcrypt (cost=10)	13	78,631977 мс
Argon2id (m=64МБ,t=3,p=2)	10	100,086245 мс

Разрыв между быстрыми и медленными алгоритмами огромен. SHA-1 и MD5 на одном ядре дают около двух миллионов хэшей в секунду, bcrypt — примерно 13, Argon2id — около 10. То есть bcrypt медленнее MD5 примерно в 160 тысяч раз при выбранных параметрах. Именно поэтому выбор алгоритма важнее, чем добавление пары символов к паролю: даже короткий пароль на bcrypt перебирать в тысячи раз дороже, чем длинный на MD5.

Второй фактор — размер пространства паролей. Шесть цифр дают миллион комбинаций. На MD5 с восемью процессами это 1,35 секунды, на bcrypt — около 5,6 часа (при 49,5 хэша/с), на Argon2id — около 19,7 часа (при 14,1 хэша/с). Для восьмисимвольного пароля из 72 символов пространство равно 72^8, то есть примерно 7,2 умножить на 10 в 14 степени. При тех же скоростях полный перебор на MD5 займёт 31 год, на bcrypt — 462 тысячи лет, на Argon2id — 1,6 миллиона лет. Это оценки для одного ноутбука и только для CPU. На видеокарте MD5 и SHA-1 ускоряются в сотни раз, поэтому

для них такие расчёты не дают никакой защиты, а bcrypt и Argon2id из-за памяти и итераций держатся заметно лучше. Дискретной видеокарты у меня нет, поэтому GPU-ускорение в работе не использовалось; вместо него сделана многопроцессность.

10. Ограничения работы

Словарь содержит всего 241 слово, и все выводы по «сложным» паролям относятся именно к нему. На промышленном словаре из миллионов строк результат был бы другим. Маска проверялась только на шестизначных числах: этого достаточно, чтобы показать механизм, но не чтобы оценить стойкость реальных паролей. Замеры сделаны на CPU без GPU и без hashcat, а John использовал 16 потоков против восьми процессов у меня. Время моего инструмента включает запуск пула процессов, поэтому на коротких словарях оно завышено. Все хэши и пароли — учебные, из методички.

11. Файлы работы

bruteforce.py — переборщик, словарная атака, маска, --lab и --selftest.

wordlist.txt — словарь из 241 пароля.

parallel_benchmark.py — замер ускорения на 1, 2, 4 и 8 процессах.

run_john.sh и папка john/ — сборка John jumbo в Docker, хэши и логи сравнения.

crack_results.csv — результаты по 16 хэшам и двум маскам.

parallel_results.csv — времена и ускорения по процессам.

john/john_results.csv — итоги сравнения с John.

hash_benchmark.py и hash_benchmark_results.csv — скорость одного хэша на одном ядре.

gen_report.py — собирает этот отчёт из перечисленных CSV.

12. Выводы

Собственный переборщик работает: он правильно определяет все четыре алгоритма, поддерживает словарь и полный перебор по маске, умеет останавливаться при находке и считает статистику. На учебном наборе из 16 хэшей словарём восстановлено 12 паролей, включая 123456, apple и Create; «сложный» пароль не найден ни одним алгоритмом, потому что его нет в словаре. Полный перебор шестизначного числа по маске занял меньше секунды на SHA-1 и MD5.

Эксперимент наглядно показывает, что скорость подбора определяется прежде всего алгоритмом. Между MD5 и bcrypt при одинаковом словаре разница в тысячи раз, а между MD5 и Argon2id — ещё больше с учётом памяти. Сложность пароля тоже важна, но она не спасает от словаря: слово Create было помечено как «очень сложное», однако нашлось сразу, потому что оно есть в списке. Настоящую стойкость даёт сочетание:

медленный алгоритм с солью, длинный уникальный пароль, ограничение числа попыток и двухфакторная аутентификация.

Многопроцессность дала ускорение в 2,6–6 раз, но не в восемь: для MD5 мешает передача кандидатов из главного процесса, для Argon2id — пропускная способность памяти. Это ожидаемый результат и хорошая иллюстрация того, что параллелизм не всегда масштабируется линейно. John the Ripper на медленных хэшах оказался в 1,9–2,4 раза быстрее за счёт 16 потоков и C-реализации, но нашёл те же самые пароли, что подтверждает корректность собственной программы.